

# Micromouse Competition 2024

## Software Guide

### 1. Introduction

#### 1.1. Overview on Micromouse competition

- Autonomous maze-solving with the shortest time to reach the center of the maze
- Multiple runs are allowed and the run with the least time will be counted
- Total time to navigate the maze is 10 minutes only

#### 1.2. Rulebook Discussion

(To be discussed in details later on)

- Mission and Rules
- Maze Size
- Robot specifications
- Penalties
- Judging Systems
- Bonuses

### 2. Timeline (2024)

#### 2.1. Research for algorithm types

30/6 - 14/7 (Week 1-2)

#### 2.2. Reviewing old repo

30/6 - 14/7 (Week 1-2)

#### 2.3. Algorithm Implementation

18/7 - 29/8 (Week 3-8)

#### 2.4. Testing on Simulation

1/9 - 9/9 (Week 5-8)

#### 2.5. PID tuning and Localization

10/9 - 25/9 (Week 9-11)

#### 2.6. Optimization of implemented functions

10/9 - 20/9 (Week 9-10)

#### 2.7. Real world testing and debugging

10/9 - 25/9 (Week 9-11)

#### 2.8. Competition Day (26/9)

### 3. Research Phase (Algorithm Selection)

#### 3.1. BFS (Breadth First Search)

- Not Optimal Solution
- High memory usage (Polynomial)

#### 3.2. DFS

- Simple
- Doesn't always get the shortest path
- 3.3. **Wall Following**
  - Simple
  - Doesn't always get the shortest path
- 3.4. **Random Search**
  - Simple
  - Doesn't always get the shortest path
- 3.5. **A\* Algorithm**
  - More efficient
  - Always gets shortest path
  - Complex and High computation
- 3.6. **Flood fill**
  - Fast and Simple
  - Effective
  - Detects dead ends

## 4. **Implementation of Algorithm**

- Final Choice was **Flood Fill** algorithm
- Written using Python first
- After logic worked correctly with python , written using c++
- After logic worked correctly with c++ , written on arduino

## 5. **Simulation**

- Left wall following / A\* / Flood fill was tried on the simulation repo
- Was used during debugging of the code
- Used during virtual testing of the code

## 6. **Sensors**

## 7. **PID**

- 7.1. **Physical Modelling**
- 7.2. **System identification (Step input & step response )**
- 7.3. **PID Tuner App for tuning the constants**
- 7.4. **Fine Tuning the constants**

## 8. **Localization**

## 9. Optimization (RTOS)

- Simplifying computational logic
- Remove any unnecessary loops and functions
- Implementing RTOS for task scheduling
- Trying RTOS on simulation to make sure it works

## 10. Testing and Debugging

- Constructing our own maze
- Testing Algorithm Code
- Serial Debugging
- Testing PID and tuning
- Record Performance for analysis and tuning
- Testing Backup Code
- Testing optimized version of code

## 11. Challenges

- Transitioning between languages
- Implementation of RTOS for the first time
- Localization
- Turning Angles

## 12. Expected Timeline (2025)

- **Research for algorithm types**  
2 Weeks (9/2 - 24/2)
- **Reviewing old repo & Rulebook**  
2 Weeks (9/2 - 24/2)

—— **Final Algorithm Decision** — (25/2) ——

- **Algorithm Implementation (Python)**  
4 Weeks (25/2 - 24/3)
- **Testing on Simulation & Debugging**  
3 Weeks (11/3 - 31/3)

— **Midterms Break** — (1/4 - 14/4) — 2 Weeks —

- **Algorithm Implementation (C++)**  
3 Weeks (15/4 - 6/5)
- **Testing on Simulation & Debugging**  
3 Weeks (15/4 - 6/5)

- **PID tuning and Localization (1)**

4 Weeks (15/4 - 13/5)

—**Finals Break** — (14/5 - 1/6) — 7 Weeks —

- **PID tuning and Localization (2)**  
2 Weeks (2/6 - 16/6)
- **Research for optimization techniques**  
1 Week (9/6 - 16/6)
- **Optimization of implemented functions**  
3 Weeks (16/6 - 7/7)
- **Algorithm Implementation (Arduino)**  
2 Weeks (8/7 - 22/7)
- **Testing on Simulation & Debugging**  
2 Weeks (15/7 - 29/7)

—— **End of Software Implementation** — (1/8) ——

- **Real world testing and debugging**

**Includes**

- **Building Maze**
- **Testing PID**
- **Smooth Turns**
- **Algorithm on Arduino**
- **Optimized Algorithm**
- **Record Performances**
- **Final Touches**

5 Weeks (1/8 - 5/9)

—— **Final Preparations** — (10/9) ——

- **Competition Day (Sept / Oct)**

## 13. **Repo and References**

### 13.1. **Repo 2024**

<https://github.com/yusufborham/Rhakotis-Micromouse/tree/main/Software>

### 13.2. **Repo 2023**

<https://github.com/zeezooX/Deshret/tree/main/src>

### 13.3. **Another repo**

<https://github.com/avishkaherath/CocoV2-Micromouse-Team-MK-MAT>

**13.4. Documentation 2023 PDF**

 AU\_Robotics\_Micromouse\_Robot\_Technical\_Documentation.pdf

**13.5. Video References**

- [The Fastest Maze-Solving Competition On Earth](#)
- Playlist :  
[https://youtube.com/playlist?list=PLAWsHzw\\_h0iiPlaGyXAr44G0XfHfyjOe7&si=i2UiAfH0M2ul13dJ](https://youtube.com/playlist?list=PLAWsHzw_h0iiPlaGyXAr44G0XfHfyjOe7&si=i2UiAfH0M2ul13dJ)

**13.6. Simulation Repo**

<https://github.com/mackorone/mms>

**13.7. PID Tuning Tutorial (Credits : Orfy)**

 MATLAB R2016a 2024-09-15 16-21-01.mp4

**13.8. Arduino on Simulation Repo**

<https://github.com/mackorone/mms-arduino>

**13.9. RTOS References**

- [RTOS Fundamentals - FreeRTOS™](#)
- [Using FreeRTOS with ESP32 and Arduino • Wolles Elektronikiste](#)
- Handout :  RTOS\_Handout.pdf

**13.10. Rulebook PDF**

 Micromouse Rulebook .pdf

**13.11. General Competition Rules PDF**

 VICTORIS 3.0 General Rules.pdf